Biblioteka <ranges> w C++

ANNA KRÓL

KAMIL ŚMIETANA

Czym jest biblioteka ranges

Biblioteka C++ Ranges Library została wprowadzona w C++20 i służy do wygodniejszej pracy z kolekcjami danych (kontenerami, tablicami, strumieniami elementów).

W prostym rozumieniu zakres jest czymś co można iterować. Jest on reprezentowany przez iterator, który oznacza początek zakresu i sentinel (z ang. wartownik), który oznacza koniec zakresu.

Zakres w standardowej bibliotece c++ (np. std::vector lub std::list) tworzy abstrakcję iteratorów w sposób, który upraszcza i wzmacnia możliwość korzystania z standardowej biblioteki szablonów (STL).

Ranges a tradycyjne iteratory

Standardowo, aby korzystać z algorytmów STL należy używać iteratorów begin() oraz end() :

```
std::vector<int> liczby{ 89, 73, 3, 93, 62, 48, 57, 99, 2, 8, 33 };
std::sort(liczby.begin(), liczby.end());
```

Teraz dzięki bibliotece <ranges> zamiast dostawać się do kontenera przez iteratory jak powyżej, wystarczy wprost podać kontener, na którym chcemy wykonać jakiś algorytm :

```
std::vector<int> liczby{ 89, 73, 3, 93, 62, 48, 57, 99, 2, 8, 33 };
std::ranges::sort(liczby);
```

Przykładowe algorytmy biblioteki ranges

- ranges::sort – sortuje elementy kontenera

```
std::vector<int> liczby{ 9,2,6,0,1,5 };
std::ranges::sort(liczby);
```

- ranges::reverse – odwraca kolejność elementów

```
std::ranges::reverse(liczby);
```

- ranges::for_each – wykonuje jakąś funkcję na wszystkich elementach

```
std::ranges::for_each(liczby, [](int i){ /* ... */ });
```

- ranges::copy – kopiuje elementy z jednego kontenera do drugiego

```
std::vector<int> liczby{ 9,2,6,0,1,5 };
std::vector<int> nowy(liczby.size());

std::ranges::copy(liczby, nowy.begin());
```

ranges::range

- ranges::range to koncept w bibliotece ranges w c++, który oznacza dowolny obiekt, po którym można iterować (jak np. wymienione wcześniej vector lub list)
- range musi z założenia udostępniać swój iterator na początek i koniec (czyli zwykłe begin() i end()), dzięki czemu można przetwarzać dane w sposób ogólny
- ranges umożliwia pisanie bardziej uniwersalnego i czytelnego kodu operującego na kolekcjach

```
std::list<int> lista_liczb = { 2, 11, 57, 40 };
// powyższy kontener spełnia koncept range, bo można po nim iterować

for (int x : lista_liczb) {
    std::cout << x << ", ";
}
```

ranges::views

- ranges::views to lekka „nakładka” na range, która nie przechowuje danych, tylko je przekształca lub filtruje w locie.
- Widoki są leniwe (lazy evaluation) – operacje wykonują się dopiero podczas iteracji.
- Pozwalają łączyć operacje w pipeline (np. filtrowanie + mapowanie) bez kopiowania danych.
-

```
std::vector<int> wektor = { 37, 25, 6, 81 };

// view obejmujący cały kontener
auto widok = std::views::all(wektor);
for (int x : widok) { std::cout << x << ", "; }
```

Range adaptors

Range adaptors to różne funkcje widoków dostępnych w bibliotece, poniżej zaprezentowano kilka najważniejszych:

- views::filter – wybiera tylko elementy spełniające określony warunek (np. tylko liczby parzyste)
- views::transform – przekształca każdy element zakresu przy pomocy funkcji (np. podniesienie każdej liczby do kwadratu)
- views::take – zwraca tylko pierwsze n elementów zakresu
- views::drop – pomija pierwsze n elementów zakresu
- views::reverse – odwraca kolejność elementów w zakresie
- views::split – dzieli zakres na podzakresy według separatora

```
std::vector<int> adaptory = { 56, 9, 17, 29, 4 };

auto widok = adaptory
    | std::views::reverse;

for (int x : widok) {
    std::cout << x << ", ";
}
```

views::iota i lazy evaluation

- views::iota(a, b) generuje zakres wartości od a do b-1
- nie tworzy kontenera (brak vector, array)
- dane powstają dopiero podczas iteracji

```
auto lazy = std::views::iota(1, 10)
    | std::views::filter([](int x) {
    std::cout << "Sprawdzam: " << x << "\n";
    return x % 2 == 0;
        });

std::cout << "Widok stworzony\n";

// Nic się jeszcze nie wykonało

for (int x : lazy) // wykonuje się dopiero tutaj
{
    std::cout << "Wynik: " << x << "\n";
}
```

Potoki (pipeline)

- Adaptory można łączyć w potoki (pipeline) przy pomocy operatora |, co pozwala budować czytelne i deklaratywne przetwarzanie danych.

```
std::vector<int> adaptory = { 56, 9, 17, 29, 4 };

auto widok = adaptory
    | std::views::filter([](int x) {return x > 15;})
    | std::views::transform([](int x) {return x * x;});

for (int x : widok) {
    std::cout << x << ", ";
}
```

W niektórych przypadkach korzystamy z wyrażenia lambda w celu przechwycenia kontekstu, jak w przykładzie powyżej

Pobranie zakresu elementów

Wykorzystując potoki oraz adaptory views::take i views::drop, można pobrać konkretny fragment kontenera.

```
std::vector<int> vec{ 5,6,2,8,9,0,1 };
auto fragment = vec | std::ranges::views::drop(4)   // od elementu o indeksie 4
                    | std::ranges::views::take(3);  // 3 elementy

for (int i : fragment)
{
    std::cout << i << " ";
}
// wynik: 9 0 1
```

DZIĘKUJEMY ZA UWAGĘ!

Przydatne linki:

- https://en.cppreference.com/w/cpp/ranges.html
- https://learn.microsoft.com/pl-pl/cpp/standard-library/ranges?view=msvc-170
- https://cpp0x.pl/artykuly/Inne-artykuly/C++-std-ranges-ranges-v3-Algorytmy-standardowe-operujace-na-zakresie/92

Materiały na githubie : Kamilo779